TestNG Class 4

Complete Beginner-to-Project Notes

Groups • XML Parameters • Browser & Environment Selection • Maven/Jenkins Execution

Purpose Turn the lecture examples into understandable, reusable automation patterns. The guide explains what each annotation or XML tag does, why it is used, how to run it, and which mistakes to avoid.

What you will learn

- Run a selected subset of tests from a large suite using TestNG groups.
- Exclude tests temporarily or permanently using a dedicated group and testng.xml.
- Pass browser and environment values to TestNG using @Parameters.
- Run the same automation class against multiple configurations such as QA + Chrome and UAT + Edge.
- Understand the difference between TestNG XML parameters and JVM/Maven system properties.
- Prepare repeatable commands suitable for local execution and CI pipelines.

Based on your uploaded “TestNG class 4.txt”. Explanatory examples and best-practice notes are added where useful; placeholder URLs and package names must be replaced with your project values.

1. Learning map and core idea

This class is about choosing which tests to run and configuring how those tests run. Imagine a project with 100 automated test methods. Running all 100 tests after every small code change can waste time. TestNG groups let you label tests; XML configuration lets you select or exclude those labels; parameters let you supply configuration such as browser and environment.

Need	TestNG feature	Example
Run only quick checks	groups + include	smoke
Run regression tests	groups + include	regression
Temporarily remove known unstable tests	groups + exclude	quarantined
Choose browser/environment from XML	@Parameters + <parameter>	browser=CHROME
Choose settings from command line/CI	JVM system properties	-Dbrowser=EDGE

A simple analogy

Treat each test like a file that can have labels. A test may carry labels such as smoke, regression, login, API, or contactPage. A run configuration says “include smoke”. TestNG selects the tests carrying that label. Another configuration can say “exclude quarantined”; those tests are removed from that run.

Important distinction A group is a label attached to tests. It does not create new test cases and does not mean tests execute in a particular order.

2. TestNG groups: what and why?

2.1 The groups attribute

The `groups` attribute on `@Test` assigns one or more group names to a test method. A method can belong to more than one group, so the same test can be part of a smoke run and a regression run.

```
import org.testng.annotations.Test;

public class LoginTests {
  @Test(groups = {"smoke", "regression", "login"})
  public void validLogin() {
    // Selenium or API test steps go here.
  }

  @Test(groups = {"regression", "login"})
  public void invalidPasswordShowsError() {
    // Verify the expected error message.
  }
}
```

Line-by-line explanation

- `@Test` tells TestNG that the method is a test method.
- `groups = {"smoke", "regression", "login"}` adds three labels to `validLogin()`; one method can match any run that includes one of those groups.

- The methods still need real actions and assertions. An empty method or a comment alone does not verify application behavior.
- Group names should be consistent across the Java annotation and XML file. Treat group naming as case-sensitive for reliable configuration: use `regression` everywhere, not `Regression` in one place.

2.2 A useful group taxonomy

Group name	Purpose	Example tests
smoke	Very fast confidence check after a build	Application opens, login works
sanity	Focused validation of a small changed area	Contact form saves a record
regression	Broader checks for existing functionality	Login, search, edit, delete
e2e	User journey across multiple components	Login → create order → payment confirmation
api	API contract/behavior tests	GET user, POST order, validation errors
ui	Browser-based checks	Form labels, navigation, buttons
quarantined	Known unstable tests temporarily excluded	Flaky test with a tracked defect

Naming recommendation Use business meaning for group names and keep a short documented taxonomy. Avoid creating a new group for every individual method unless there is a real selection need.

3. How to run only a selected group

3.1 Example class with multiple groups

```
public class ContactAndDealsTests {
  @Test(groups = {"regression", "sanity", "login", "contactPage"})
  public void testCases1() { /* login/contact quick check */ }

  @Test(groups = {"regression", "e2e", "contactPage"})
  public void testCases2() { /* create contact workflow */ }

  @Test(groups = {"sanity", "e2e", "dealsPage"})
  public void testCases3() { /* create deal workflow */ }

  @Test(groups = {"regression", "e2e"})
  public void testCases4() { /* edit/delete flow */ }

  @Test(groups = {"e2e", "dealsPage"})
  public void testCases5() { /* longer deal journey */ }
}
```

For this sample, an include of `regression` selects testCases1, testCases2, and testCases4. An include of `sanity` selects testCases1 and testCases3. An include of `e2e` selects testCases2, testCases3, testCases4, and testCases5.

3.2 Option A: IDE Run Configuration

1. Right-click the test class or project in Eclipse.
2. Choose Run As → TestNG Test (or Run Configurations, depending on the Eclipse/TestNG plugin version).
3. Open the Groups / Groups of Tests selector and choose the group you want, such as `regression`.
4. Apply and Run. Check the TestNG Results view to confirm which methods ran.

The menu labels can differ slightly by Eclipse and TestNG plugin version. If group selection is unavailable, use the XML suite approach below, which is also suitable for CI.

3.3 Option B: select groups in testng.xml

XML: include only regression tests

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Regression Suite">
  <test name="Regression Tests">
    <groups>
      <run>
        <include name="regression"/>
      </run>
    </groups>
    <classes>
      <class name="tutorial1.ContactAndDealsTests"/>
    </classes>
  </test>
</suite>
```

How to read this XML

- `<suite>` is the top-level TestNG execution suite.
- `<test>` defines one TestNG test section and can include one or more Java classes.
- `<groups><run><include .../></run></groups>` selects methods assigned to the named group.
- `<class name="..."/>` must use the fully qualified class name: package name + class name.

Multiple groups Multiple `<include>` tags select tests in any of those groups. For example, include `sanity` and include `regression` to run tests in either group. A method in both groups is still one test method; it is not conceptually a duplicate test just because it matches both labels.

Select sanity OR regression

```
<groups>
  <run>
    <include name="sanity"/>
    <include name="regression"/>
  </run>
</groups>
```

4. How to skip or exclude a subset of tests

4.1 Assign a common exclusion group

If 10 out of 100 tests should be excluded from a particular run, tag those methods with a common group such as `quarantined` or `skip`. The test methods remain in the code; the XML configuration decides not to execute that group in the selected run.

Example: label a test for temporary quarantine

```
@Test(groups = {"regression", "quarantined"})
public void knownFlakyPaymentTest() {
   // Keep the test code and its expected behavior.
   // Track the reason and defect/issue outside this snippet.
}
```

4.2 Exclude that group using XML

Exclude quarantined tests

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Project Suite">
  <test name="Regression without quarantine">
    <groups>
      <run>
        <exclude name="quarantined"/>
      </run>
    </groups>
    <classes>
      <class name="tutorial1.ContactAndDealsTests"/>
    </classes>
  </test>
</suite>
```

With this run configuration, regular regression tests still run, but the methods marked `quarantined` are left out even if they also belong to `regression`.

4.3 Exclude versus disable

Approach	What it means	Best use
XML `<exclude>`	Skip a group for this particular suite/run configuration	Choose different subsets without editing Java code
`@Test(enabled = false)`	Disable a test method in the code by default	A test is intentionally disabled and the team accepts that repository-level decision
`@Ignore`	Ignore a method/class depending on how it is applied	Use sparingly; visibility/reason may be less explicit in a test plan
Group such as `quarantined`	Classify tests so a run can exclude them	Temporary isolation with issue owner and re-enable date

Quality safeguard Do not silently exclude failing tests just to make a build green. Record why they are quarantined, link a defect, assign an owner, and review the exclusion regularly.

5. TestNG parameterization: the basic idea

Parameterization means supplying configuration values to a test or setup method from outside the Java method. In this class, XML parameters are used to select a browser or environment without hardcoding those values into the method body.

5.1 One parameter: browser

```
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class BrowserConfigTest {
   @Test
   @Parameters({"browser"})
   public void testCases1(String browser) {
      System.out.println("Executing tests on " + browser);
   }
}
```

`@Parameters({"browser"})` says that TestNG should inject an XML parameter called `browser` into the method argument. The Java parameter `String browser` receives the configured string value, for example `Chrome` or `Edge`.

5.2 Define the value in testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Browser Suite">
   <parameter name="browser" value="Edge"/>
   <test name="Regression">
      <classes>
         <class name="tutorial1.BrowserConfigTest"/>
      </classes>
   </test>
</suite>
```

The parameter name must match: XML `name="browser"` pairs with `@Parameters({"browser"})`. If they differ, TestNG cannot inject the expected value and setup fails before the test executes.

Scope A parameter declared under `<suite>` can be available to test sections in that suite. A parameter declared inside a specific `<test>` section applies to that TestNG test section and is useful when different sections need different values.

6. Browser and environment combinations

The lecture lists combinations such as QA + Chrome, QA + Edge, UAT + Chrome, and UAT + headless browser. A single test class can be run repeatedly with different XML configuration values, as long as setup code uses those values to choose the target URL and browser.

6.1 Configure different values in separate <test> sections

Same class, separate configurations

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Cross Browser Suite">
  <test name="QA Chrome Smoke">
    <parameter name="environment" value="QA"/>
    <parameter name="browser" value="CHROME"/>
    <groups><run><include name="smoke"/></run></groups>
    <classes><class name="tutorial1.SauceDemoTest"/></classes>
  </test>

  <test name="UAT Edge Regression">
    <parameter name="environment" value="UAT"/>
    <parameter name="browser" value="EDGE"/>
    <groups><run><include name="regression"/></run></groups>
    <classes><class name="tutorial1.SauceDemoTest"/></classes>
  </test>
</suite>
```

Here, each `<test>` section supplies a different environment and browser to the test class. The group filter for each section can differ too. TestNG configuration is not an environment deployment; it only tells the test code where and how to run.

6.2 A Selenium setup example

The following is a template, not a turnkey test against a real application. Replace the example QA and UAT URLs with URLs authorized for your project, and add the real assertions/locators for your application.

Part 1: imports, browser selection, and setup

```
package tutorial1;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class SauceDemoTest {
  private WebDriver driver;

  @BeforeMethod
  @Parameters({"environment", "browser"})
  public void setUp(String environment, String browser) {
    switch (browser.toLowerCase()) {
      case "chrome" -> driver = new ChromeDriver();
      case "edge" -> driver = new EdgeDriver();
      case "headless-chrome" -> {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--headless=new");
        driver = new ChromeDriver(options);
      }
      case "headless-edge" -> {
        EdgeOptions options = new EdgeOptions();
        options.addArguments("--headless=new");
        driver = new EdgeDriver(options);
      }
      default -> throw new IllegalArgumentException(
        "Unsupported browser: " + browser);
    }

    driver.manage().timeouts().implicitlyWait(Duration.ZERO);
    driver.manage().window().maximize();
    driver.get(baseUrlFor(environment));
  }
```

Part 2: environment mapping, assertion, and cleanup

```
private String baseUrlFor(String environment) {
    return switch (environment.toLowerCase()) {
        case "qa" -> "https://qa.example.test"; // replace this URL
        case "uat" -> "https://uat.example.test"; // replace this URL
        default -> throw new IllegalArgumentException(
            "Unsupported environment: " + environment);
    };
}

@Test(groups = {"smoke", "login"})
public void applicationPageLoads() {
    Assert.assertFalse(driver.getTitle().isBlank(),
        "Page title should not be blank");
}

@AfterMethod(alwaysRun = true)
public void tearDown() {
    if (driver != null) {
        driver.quit();
        driver = null;
    }
}
}
```

What this code is doing

- `WebDriver driver` stores the browser session for this test instance.
- `@BeforeMethod` runs before each test method. The `@Parameters` annotation injects the environment and browser values for that TestNG XML `<test>` section.
- The `switch` selects Chrome, Edge, or a headless option. An unknown browser fails fast with a clear error rather than silently choosing a default.
- `baseUrlFor()` maps the environment name to a base URL. Replace the sample `.test` URLs with your QA/UAT URLs.
- `implicitlyWait(Duration.ZERO)` avoids mixing long implicit waits with explicit waits; use `WebDriverWait` for specific conditions in the actual test.
- The assertion checks a simple page-load condition. A real test should validate meaningful expected behavior such as a heading, a logged-in user, or a successful record creation.
- `@AfterMethod(alwaysRun = true)` attempts to close the browser after each test method even if a prior test/setup phase failed. `quit()` closes the browser session.

Version note The arrow-style `switch` syntax shown is supported in modern Java (Java 14+ as a standard feature). If your project targets older Java, use traditional `switch` statements or `if/else` blocks.

7. TestNG XML parameters vs command-line properties

This is a common source of confusion. `@Parameters` reads TestNG parameters from TestNG configuration (such as `testng.xml`). A Maven command-line system property like `-Dbrowser=EDGE` is a JVM/system property; do not assume it automatically overrides the XML value injected through `@Parameters`. Choose one configuration pattern or deliberately bridge between them.

Pattern A: use TestNG XML parameters

- Use `@Parameters({"environment", "browser"})` on the setup method.

- Define matching `<parameter name="environment" value="QA"/>` and `<parameter name="browser" value="CHROME"/>` entries in XML.
- Run Maven with a POM configured to use the suite XML, for example `mvn test`. Values come from the XML file.

Pattern B: use JVM system properties for CLI/CI overrides

Use this pattern when you want a command such as `mvn test -Denvironment=QA -Dbrowser=CHROME` to set the values. In Java, read those system properties using `System.getProperty`, with defaults and validation. Configure Maven Surefire to pass the values to the test JVM.

```
String environment = System.getProperty("environment", "QA");
String browser = System.getProperty("browser", "CHROME");
// Validate both values before creating the driver.
```

For this pattern, remove the matching `@Parameters` injection for those values or make your configuration strategy explicit. Avoid having XML, environment variables, and JVM properties all competing to supply the same setting without a clear precedence rule.

7.1 Example Maven Surefire configuration

pom.xml: connect Maven values to the test JVM

```
<properties>
  <environment>QA</environment>
  <browser>CHROME</browser>
</properties>

<build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-surefire-plugin</artifactId>
   <configuration>
    <suiteXmlFiles>
     <suiteXmlFile>testng.xml</suiteXmlFile>
    </suiteXmlFiles>
    <systemPropertyVariables>
     <environment>${environment}</environment>
     <browser>${browser}</browser>
    </systemPropertyVariables>
   </configuration>
  </plugin>
 </plugins>
</build>
```

Run from a terminal in the project root:

```
mvn test -Denvironment=QA -Dbrowser=CHROME
mvn test -Denvironment=UAT -Dbrowser=EDGE
```

The Maven properties override the defaults in `<properties>`; Surefire makes them available to Java through `System.getProperty`. Make sure the class uses Pattern B for these values. This is separate from TestNG XML `@Parameters`.

8. Running groups and configurations in CI/Jenkins

8.1 Practical CI workflow

1. A developer opens a pull request. CI runs the `smoke` group on the default supported browser.
2. A merge to the main branch triggers a broader `regression` run.
3. A nightly pipeline runs `regression` and `e2e` across a larger browser/environment matrix.
4. Known unstable tests are labelled `quarantined`, excluded only with a visible reason and follow-up issue.

8.2 Example command-line usage

If Maven Surefire is already configured to run `testng.xml`, and the Java code reads system properties as Pattern B above, a Jenkins shell step can run:

```
mvn clean test -Denvironment=QA -Dbrowser=CHROME
```

A separate TestNG XML suite can select the groups to run. For example, `testng-smoke.xml` includes `smoke`, while `testng-regression.xml` includes `regression`. If your POM chooses the suite XML dynamically, configure that property explicitly for the Maven Surefire version and project setup in use.

8.3 Jenkins parameters

Jenkins parameter	Sample value	How the test uses it
ENVIRONMENT	QA / UAT	Selects the base URL or config profile
BROWSER	CHROME / EDGE / HEADLESS-CHROME	Selects WebDriver implementation/options
SUITE	smoke / regression	Chooses a TestNG XML file or suite strategy

Secrets Do not store passwords, tokens, or production credentials in testng.xml, source code, or ordinary build parameters. Store secrets in Jenkins Credentials and inject them only into the required process/environment.

9. Common mistakes and troubleshooting

Problem	Likely cause	Fix
No tests execute after using include	Group name misspelled, case mismatch, wrong class path	Compare annotation/XML names exactly; check the fully qualified class name
Tests outside target group execute	Groups are added incorrectly or XML filter is missing/not loaded	Confirm Maven/IDE is running the intended testng.xml
Expected group seems skipped	Method not annotated with `@Test` or method belongs to a different group	Check the annotation and actual selected suite
`@Parameters` cannot find value	XML parameter missing or name mismatch	Match `@Parameters({"browser"})` to `<parameter name="browser" .../>`

Problem	Likely cause	Fix
CLI `-Dbrowser=EDGE` has no effect	Code reads `@Parameters`, not a JVM system property	Use XML parameters or switch to `System.getProperty` with Surefire configuration
Unsupported browser exception	Value spelling differs, for example `headlessChrome` vs `headless-chrome`	Standardize accepted values and validate early
CI test fails but local passes	Different browser version, URL, permissions, environment data, or timing	Log effective config safely; align dependencies and use condition-based waits
Quarantined tests stay excluded forever	No owner or review date	Track a defect, owner, and re-enable target date

Debug checklist

- Confirm the intended suite XML file is actually being executed.
- Print the effective browser and environment at startup, but never print secrets.
- Check group names in both annotations and XML for spelling/case differences.
- Check the package declaration and fully qualified class name in XML.
- Ensure the relevant Maven Surefire plugin configuration is being used.
- Capture a screenshot and browser logs for genuine UI failures; do not add long fixed sleeps as the first fix.

10. Best practices for maintainable automation

Groups

- Use a small, documented set of stable group names such as `smoke`, `sanity`, `regression`, `api`, `ui`, `e2e`, and `quarantined`.
- A test may have several groups, but only add labels that reflect how the test will be selected.
- Keep group filters in version-controlled XML files for repeatable local and CI runs.
- Do not use groups to enforce dependencies or order. Use explicit dependencies only when tests genuinely depend on each other; preferably make tests independent.
- Do not permanently hide red tests with an exclude. Track the defect and remove the exclusion when fixed.

Parameters and environments

- Keep configuration out of test logic where practical. Use parameters/properties to control browser and base URL.
- Validate environment and browser values at startup; fail with a clear error if unsupported.
- Never hardcode private credentials in Java or XML.
- Use a clearly defined precedence. For example: Maven/JVM property override first, then a safe default; avoid ambiguous mixed sources.
- Keep QA/UAT data isolated. A cleanup routine should not delete shared data unless the test owns that data.

Selenium execution

- Use `WebDriverWait` for specific conditions such as visibility or clickability instead of long `Thread.sleep()` calls.

- Create and quit browser sessions at the correct lifecycle boundary. For independent tests, `@BeforeMethod` and `@AfterMethod` are a common default.
- If enabling parallel execution, do not share a mutable WebDriver instance across threads. Use per-thread driver management such as `ThreadLocal` only when your framework is designed for it.
- Use meaningful assertions so a test can fail for a reason that matters to the user.
- Keep URLs and test accounts configured per environment; never point destructive tests at production by accident.

11. Hands-on exercises

Exercise 1: pick smoke tests

Create six methods: valid login, invalid login, search contact, create contact, delete contact, and long end-to-end deal journey. Assign groups so valid login and search contact are `smoke`; all six are `regression`; the long journey is `e2e`. Write a testng.xml file that selects only `smoke`.

Exercise 2: exclude quarantine

Mark two unstable methods with the `quarantined` group while leaving them in `regression`. Configure a suite that includes `regression` and excludes `quarantined`. Predict which methods should execute before you run it.

Exercise 3: browser + environment configuration

Run the same test class with QA/Chrome and UAT/Edge. Use either the XML `@Parameters` approach or the JVM system-property approach; do not mix the two without a precedence rule. Print the resolved configuration at startup and assert that the intended base URL loaded.

Suggested answers

- Exercise 1: place `<include name="smoke"/>` inside `<groups><run>` in the relevant `<test>` block.
- Exercise 2: combine `<include name="regression"/>` and `<exclude name="quarantined"/>`; the exclusion should remove matching tests from that run.
- Exercise 3: provide `environment` and `browser` values using the selected configuration pattern, map each environment to an approved URL, create the correct WebDriver, and close it in teardown.

12. Interview revision questions

Q. What is a TestNG group?

A. A label assigned to test methods so a selected category can be included or excluded in a run.

Q. Can one test belong to multiple groups?

A. Yes. The groups attribute accepts an array of names.

Q. How do you run only regression tests?

A. Tag them with `regression`, then use an IDE group filter or `<include name="regression"/>` in testng.xml.

Q. How do you skip 10 out of 100 tests without deleting them?

A. Assign a common group and exclude it in the suite configuration, ideally with a tracked reason and owner.

Q. What is the difference between include and exclude?

A. Include selects tests matching a group; exclude removes tests matching a group from that run.

Q. What is `@Parameters` used for?

A. It injects named values configured in TestNG XML into annotated test or configuration methods.

Q. Why might `-Dbrowser=EDGE` not affect `@Parameters("browser")`?

A. A JVM system property and a TestNG XML parameter are different configuration mechanisms; code must read the appropriate source.

Q. What is the difference between `@Parameters` and `@DataProvider`?

A. `@Parameters` commonly supplies suite/test configuration; `@DataProvider` supplies multiple rows of test data to repeat a test with different inputs.

Q. How should Jenkins select a browser?

A. Use a validated parameter or configuration value, map it to a supported driver, and keep secrets in a credential store.

Q. Why is a quarantined group risky?

A. It can hide real defects if it is not tracked, reviewed, and eventually removed.

13. Quick reference

Goal	Where to configure	Remember
Run one category	`@Test(groups = {"smoke"})` + XML include	Names must match
Run multiple categories	Multiple `<include>` elements	Methods matching either group can run
Exclude a category	XML `<exclude name="quarantined"/>`	Use for one run configuration; track why
Supply browser/env via XML	`<parameter>` + `@Parameters`	Names must match exactly
Supply values with Maven CLI	`-D...` + `System.getProperty` + Surefire	Not automatically the same as TestNG XML parameters
Run browser headlessly	Browser options in driver setup	Use valid browser-specific options
Run in Jenkins	Version-controlled suite + validated parameters	Keep secrets in credentials

Final takeaway Use groups to decide WHAT runs. Use parameters or system properties to decide HOW and WHERE it runs. Keep these responsibilities separate, make the configuration reproducible, and never hide test failures without accountability.